LOS NÁUFRAGOS · ARQUITECTURA DE COMPUTADORES · SEXTO SEMESTRE

Núcleo de planificación

Los algoritmos de planificación de CPU —FCFS, SJF, SRTF y Round Robin— implementados en C. Cada llamada devuelve la línea de tiempo completa de la ejecución y las métricas por proceso: es lo que el simulador dibuja.

Biblioteca: `libplanificador.so` · se integra en `planificador-service`

Equipo Los Náufragos · Kamila Marín Cossio · Juan Fernando Muñoz Silva

---

El sistema del que hace parte

Este entregable no vive solo: es el núcleo de cómputo de un sistema completo que se construye en paralelo desde Programación en Ambiente Web 2.

El sistema completo recibe un conjunto de procesos con su tiempo de llegada y su ráfaga, corre un algoritmo de planificación y dibuja el diagrama de ejecución con las métricas de espera, retorno y respuesta. Esta biblioteca produce esa línea de tiempo. El frontend la dibuja; nada más la calcula.

```
planificador.c → gcc -shared -fPIC -O2 → libplanificador.so → imagen Docker → interfaz JNA → planificador-service → POST /planificaciones → Angular
```

El frente de Web 2 —equipo **Los Náufragos**— construye la aplicación completa: el frontend en Angular, los microservicios, el modelo de datos y la autenticación. Este documento cubre **únicamente** lo que corresponde a Arquitectura de Computadores.

Qué le corresponde a este equipo

Delimitar lo que no se pide es tan importante como lo que sí. Lo de la derecha no está prohibido: simplemente no se evalúa aquí, y consume el tiempo que exige lo de la izquierda.

QUÉ ENTRA

- FCFS, SJF, SRTF y Round Robin en C
- La línea de tiempo completa, con los intervalos de CPU ociosa

- Métricas por proceso: espera, retorno y respuesta
- Los promedios de las tres métricas
- La interfaz JNA y el `@Service` que la envuelve
- La implementación en Java, como línea base
- El `Dockerfile` y el manifiesto de despliegue

QUÉ NO ENTRA

- El frontend en Angular y el diagrama de Gantt
- La autenticación y la autorización
- El catálogo de conjuntos de procesos y el historial
- Planificación multinivel o con envejecimiento
- Operaciones de entrada y salida: los procesos son solo CPU
- Los escenarios de calidad del frente de Web 2

> **La autenticación y la autorización no se evalúan en este curso.** El sistema completo exige OAuth 2.0 con Keycloak, pero configurarlo y demostrarlo es responsabilidad del frente de Web 2. El servicio que expone esta biblioteca queda protegido igual que los demás, y basta con no romper esa configuración.

La biblioteca: qué debe hacer

Cada requisito trae su criterio de aceptación, que es la forma de saber si quedó hecho sin discutirlo. Todo se implementa en C y se compila como biblioteca compartida de Linux.

> **RN-01** **FCFS**
>
> Primero en llegar, primero en ser atendido. Sin expropiación. Con la CPU ociosa marcada explícitamente cuando ningún proceso ha llegado todavía.
>
> CRITERIO Para tres procesos que llegan en 0, 2 y 4 con ráfagas 5, 3 y 1, la línea de tiempo coincide exactamente con la dibujada a mano, incluida la rebanada ociosa si la hay.

RN-02 **SJF sin expropiación**

De los procesos que ya llegaron, se escoge el de ráfaga más corta. Una vez en CPU, termina.

CRITERIO Para el conjunto de referencia, el promedio de espera es menor que el de FCFS, y coincide con el valor calculado a mano.

RN-03 **SRTF con expropiación**

El proceso en CPU se interrumpe si llega uno con tiempo restante menor. Es el que genera más rebanadas y el que más errores esconde.

CRITERIO Existe un conjunto donde un proceso aparece en **tres rebanadas separadas** de la línea de tiempo, y las tres coinciden con el cálculo a mano.

RN-04 **Round Robin**

Turnos de duración fija. El quantum entra como parámetro. La cola es circular y el orden de reingreso importa.

CRITERIO Con quantum 2 sobre el conjunto de referencia, la secuencia de rebanadas coincide con la calculada a mano. Con quantum mayor que la ráfaga más larga, el resultado es idéntico al de FCFS.

RN-05 **Métricas por proceso y promedios**

Espera, retorno y respuesta de cada proceso, más los tres promedios. La respuesta es el primer momento en CPU, no el último.

CRITERIO Para los conjuntos de referencia, las tres métricas de cada proceso coinciden exactamente con las calculadas a mano. Cero tolerancia: son enteros.

RN-06 **Catálogo de algoritmos publicado por la biblioteca**

`listar_algoritmos` devuelve los identificadores disponibles. Agregar un algoritmo no debe obligar a tocar Java.

CRITERIO Se agrega un quinto algoritmo al código en C, se recompila, y aparece en la interfaz sin modificar ni un archivo `.java`. El `git diff --stat` lo demuestra.

El contrato con el frente de Web 2

Lo que une los dos entregables es una interfaz, no una reunión. Se acuerda en la primera semana, se congela, y queda escrita en la bitácora de los dos frentes.

La cabecera en C, que es el documento de referencia:

```
/* planificador.h */

typedef struct { int pid; int llegada; int rafaga; int prioridad; } Proceso;
typedef struct { int pid; int inicio; int fin; }  Rebanada;   /* pid = -1 → CPU ociosa */
typedef struct { int pid; int espera; int retorno; int respuesta; } Metrica;

typedef struct {
    Rebanada rebanadas[MAX_REBANADAS];  int n_rebanadas;
    Metrica  metricas[MAX_PROCESOS];    int n_metricas;
    double   espera_prom, retorno_prom, respuesta_prom;
} Resultado;

/* Devuelve 0 si todo salio bien, o un codigo de error negativo. */
int planificar(const char *algoritmo, const Proceso *procs, int n,
               int quantum, Resultado *salida);

/* Los identificadores disponibles, separados por coma. */
int listar_algoritmos(char *buffer, int tam);
```

Y la interfaz JNA del lado de Java. No hay que escribir código puente: se declara la firma y la biblioteca queda disponible como si fuera un objeto más.

```
public interface LibPlanificador extends Library {
    LibPlanificador INSTANCE = Native.load("planificador", LibPlanificador.class);

    int planificar(String algoritmo, Proceso.ByReference procs, int n,
                   int quantum, Resultado.ByReference salida);

    int listar_algoritmos(byte[] buffer, int tam);
}

@Service
public class PlanificadorNativo {
    public ResultadoDTO ejecutar(String algoritmo, List<ProcesoDTO> procesos, int quantum) {
        // mapear a la estructura, invocar, traducir el resultado a DTO
    }
}
```

`listar_algoritmos` no es un adorno: es lo que permite agregar un quinto algoritmo sin tocar Java. Si la lista de algoritmos disponibles la publica la biblioteca, el frente de Web 2 la consulta y la muestra; si estuviera escrita en un `enum` de Java, agregar un algoritmo obligaría a modificar los dos lados y a redesplegar los dos. Esa decisión es la que sostiene el escenario de modificabilidad del proyecto completo.

> **Tres cosas deben quedar fijadas por escrito, y no son negociables después.**
>
> **1. Quién libera la memoria.** C no tiene recolector. Si la biblioteca reserva, debe exponer una función para liberar, y el lado de Java tiene que llamarla — típicamente en un `try/finally`. Si en cambio Java preasigna los arreglos y C solo los llena, hay que decirlo, porque entonces Java debe saber cuánto reservar antes de llamar.
>
> **2. Cómo se reporta un error.** C no lanza excepciones: devuelve códigos. La tabla de códigos es parte del contrato, y el lado de Java los traduce a excepciones o a respuestas HTTP. Un código no documentado que aparece en producción es un fallo del contrato, no del código.
>
> **3. Los casos de referencia.** Un conjunto de entradas con su salida esperada, escritas **antes** de implementar. Sirven de oráculo a los dos lados: conjuntos de tres a cinco procesos cuyo diagrama de ejecución y cuyas métricas se calculan a mano en papel, más la implementación en Java. Son enteros: la coincidencia debe ser exacta.

Requisitos de calidad

Un requisito de calidad no es un adjetivo: es un escenario con seis partes y un número que se puede comprobar. Los nombres son los de Len Bass, *Software Architecture in Practice*. Los tres de abajo son los que corresponden a este frente, y son distintos de los que se le exigen al frente de Web 2.

RQ-01 · DESEMPEÑO · ATRIBUTO ASIGNADO

Es la razón de existir del componente nativo, y la que hay que demostrar con números. La pregunta no es "¿es rápido?", sino "¿cuánto más rápido que la misma cosa en Java, y cuánto costó?".

FUENTE	El equipo, ejecutando el banco de medición.
ESTÍMULO	Se planifican **10.000 procesos** con SRTF —el algoritmo más costoso, porque reevalúa en cada unidad de tiempo—, primero en C y luego en Java.
ARTEFACTO	`libplanificador.so` invocada por JNA, y la implementación equivalente en Java.
ENTORNO	Contenedor en operación normal, una sola petición a la vez, con los límites declarados en el manifiesto.
RESPUESTA	Las dos versiones devuelven la misma línea de tiempo y las mismas métricas, y se registra el tiempo de cada una.
MEDIDA	**Cinco corridas de cada lado, reportando la mediana**, más la relación entre las dos. La corrida completa responde en menos de **1 segundo**.

CÓMO SE MIDE

Un banco de medición en Java con las dos variantes sobre el mismo conjunto de procesos, descartando las tres primeras corridas para que el JIT se caliente. Los conjuntos grandes se generan con un script y semilla fija, no a mano, para que la medición sea repetible.

Si C no resulta más rápido, eso se reporta y se explica. Una medición negativa bien hecha vale más que una ganancia inventada, y es un resultado perfectamente posible: cruzar la frontera Java↔C cuesta, y si se cruza demasiadas veces ese costo se come la ganancia. Entender *por qué* pasó es exactamente el aprendizaje del curso.

RQ-02 · TESTABILIDAD

El frente nativo debe poder demostrar que su parte está bien **sin depender del sistema completo**. Si para probar la biblioteca hay que levantar Kubernetes, abrir el navegador y esperar al otro equipo, no se va a probar.

FUENTE	Cualquier integrante del equipo, tras modificar un algoritmo.
ESTÍMULO	Se pide comprobar que los cuatro algoritmos siguen produciendo la línea de tiempo correcta.
ARTEFACTO	La suite de pruebas de `libplanificador.so`, ejecutada contra la biblioteca compilada.
ENTORNO	Un equipo de desarrollo cualquiera, con el repositorio recién clonado y **sin el sistema desplegado**.
RESPUESTA	Un solo comando compila y ejecuta los casos de referencia, reportando cuáles pasaron con el valor esperado y el obtenido.
MEDIDA	La suite corre en **menos de 30 segundos**, cubre los **cuatro algoritmos** con al menos dos conjuntos cada uno, y compara la línea de tiempo completa, no solo los promedios.

> CÓMO SE MIDE
>
> **Se clona el repositorio en una carpeta limpia y se ejecuta el comando único. La comparación es sobre la secuencia de rebanadas, no sobre los promedios: dos líneas de tiempo distintas pueden dar el mismo promedio de espera, así que comparar solo el promedio deja pasar errores reales.**

RQ-03 · FACILIDAD DE DESPLIEGUE

Este atributo entra por una razón concreta: incorporar una biblioteca compilada a un contenedor cambia cómo se construye y cómo se despliega el sistema. Es el aporte de este frente al proyecto completo, y por eso se mide.

FUENTE	Quien evalúa el proyecto, o cualquier integrante en una máquina nueva.
ESTÍMULO	Se despliega el sistema completo desde el repositorio recién clonado.
ARTEFACTO	El `Dockerfile` multietapa de `planificador-service` y el manifiesto de despliegue.
ENTORNO	Máquina limpia con Docker instalado y nada más: sin `gcc`, sin JDK, sin artefactos en caché.
RESPUESTA	Un solo comando construye las imágenes y levanta los servicios. El sistema queda listo sin intervención manual.
MEDIDA	La imagen final pesa **menos de 250 MB** y no contiene `gcc`. El sistema responde su primera planificación correcta en **menos de 90 segundos**.

CÓMO SE MIDE

`docker images` para el tamaño, y el tiempo desde `docker compose up` o `kubectl apply` hasta que el servicio responde su primera petición correcta. Tres corridas de cada una, reportando la mediana, con la imagen construida desde cero (`--no-cache`) para que el resultado no dependa de capas ya en el disco.

El despliegue a producción

Además del componente nativo, este equipo responde por llevar la aplicación completa a producción. Sale naturalmente de quien construye la biblioteca, porque es quien tiene que incorporar un artefacto compilado a una imagen.

01

El `Dockerfile` multietapa

Una etapa compila `libplanificador.so` con `gcc`; la siguiente copia solo la biblioteca y el JAR. La imagen final no lleva compilador ni fuentes en C. Si la imagen final pesa lo mismo que la de construcción, el multietapa no está haciendo nada.

02

El manifiesto de despliegue de todo el sistema

El YAML de Kubernetes o el archivo de stack de Docker Swarm de *todos* los servicios, no solo del nativo, con los límites de CPU y memoria declarados explícitamente. Es también el único lugar donde se declara el entorno que los escenarios de calidad necesitan para poder comprobarse.

03

El arranque desde cero, comprobado

Un solo comando levanta el sistema en una máquina limpia, sin pasos manuales y sin que nadie tenga que conocer el orden. Se comprueba en una máquina donde el proyecto nunca se había levantado — no en el portátil donde se construyó.

04

Cómo se carga la biblioteca dentro del contenedor

Dónde queda el `.so`, cómo lo encuentra JNA — `java.library.path`, `LD_LIBRARY_PATH` o `Native.load` con ruta absoluta— y qué pasa si no está. Esa decisión se registra en la bitácora: es la causa más común de que un sistema funcione en el portátil y no en el contenedor.

Entregables

Qué se entrega y en qué forma. Lo que no está aquí, no se pide.

01

El código fuente en C

En el mismo repositorio del proyecto, en su propia carpeta, con un `Makefile` o un script que lo compile con un comando. Compila sin advertencias con `-Wall`.

02

La interfaz JNA y el servicio que la envuelve

La interfaz Java con las firmas, las estructuras mapeadas, y el `@Service` de Spring que traduce entre el modelo del dominio y los tipos que C entiende.

03

La suite de pruebas del componente nativo

Los casos de referencia con su valor esperado, ejecutables con un comando y sin levantar el sistema. El reporte en verde es la evidencia de RQ-02.

04

Las tres mediciones

La tabla de C contra Java para RQ-01; el tiempo de la suite para RQ-02; el tamaño de imagen y el tiempo de arranque antes y después para RQ-03. Datos crudos, no solo la conclusión.

05

El manifiesto de despliegue y el `Dockerfile`

Versionados en el repositorio, con el README que diga cuál es el comando que levanta todo desde cero.

06

La bitácora de decisiones

Una tabla con **fecha**, **decisión**, **alternativas descartadas**, **por qué se descartaron** y **contra qué requisito de calidad se justificó**. Una decisión sin alternativa evaluada es una decisión que no se tomó.

Lo que se evalúa no es que compile, sino que se sepa por qué está ahí. Qué gana el sistema al bajar ese cálculo a código nativo, qué cuesta en complejidad de despliegue, y cómo cambia el tamaño de la imagen y el arranque del contenedor. Un componente que funciona pero cuya decisión no se puede defender con una medición no cumple el requisito.

Por dónde empezar

Cinco tareas para la primera semana, en este orden.

01

Acordar y congelar el contrato con el frente de Web 2

Antes de escribir una línea de C. Las firmas, quién libera la memoria, los códigos de error y los casos de referencia. Queda escrito en la bitácora de los dos frentes. Cambiarlo después cuesta el doble.

02

Escribir los casos de referencia con su valor esperado

Antes de implementar, no después. Sirven conjuntos de tres a cinco procesos cuyo diagrama se dibuja a mano en papel. Escribirlos después es confirmar lo que salió, no comprobar que está bien.

03

La biblioteca en C, probada desde C

Un `main` pequeño que la ejercita con los casos de referencia y compara. Sin Java, sin Spring, sin contenedor. Si aquí falla, no tiene sentido seguir subiendo.

04

La interfaz JNA, con un caso de punta a punta

Un solo caso que entre por Java y salga por C con el valor correcto. Es donde aparecen los errores de mapeo de tipos y de orden de campos en las estructuras — no en el último día, sino ahora.

05

El `Dockerfile` multietapa y la primera medición

Incorporarla al contenedor y medir de inmediato el tamaño de imagen y el arranque, con y sin la biblioteca. Es el dato de RQ-03 y se pierde si se deja para el final, porque ya no hay un "antes" con qué comparar.

Este documento cubre el componente de Arquitectura de Computadores del proyecto **Planificador de procesos**. El enunciado del sistema completo, con sus requisitos funcionales, sus escenarios de calidad y el bloque de autenticación, está en el documento del equipo Los Náufragos de Programación en Ambiente Web 2.